

WAREHOUSE FUSION

Système Unifié de Navigation Autonome et de Manipulateur
Robotique par Apprentissage par Renforcement

Rapport Technique Intégral

Intégration de ROS 2 Nav2, Robosuite, MuJoCo, PyTorch et Stable-Baselines3

Auteur : Rabeb

Projet : Warehouse Fusion

Robot : Franka Emika Panda & Mobile
Trailer

Environnement : Linux (Ubuntu 22.04)

Framework : ROS 2 Humble / Robosuite

Date : Septembre 2026

Table des matières

1 Résumé Exécutif et Objectifs du Projet

Le projet **Warehouse Fusion** s'inscrit dans le domaine de la robotique d'entrepôt de nouvelle génération. L'objectif principal est d'orchestrer la coopération autonome entre un **robot mobile à remorque (Trailer)** gérant la logistique horizontale et un **bras manipulateur (Franka Emika Panda)** assurant la saisie et le transbordement d'objets (Pick & Place).

1.1 Objectifs Principaux

1. **Navigation Autonome (Nav2)** : Guider le chariot mobile à travers l'entrepôt vers la station de chargement du bras Panda en évitant les obstacles et en assurant un amarrage précis ($x = -0.24m, y = 2.80m$).
2. **Apprentissage par Renforcement (RL)** : Entraîner une politique sous **Robosuite** à l'aide de l'algorithme **PPO (Proximal Policy Optimization)** pour permettre au bras Panda d'intercepter, d'attraper et de soulever l'objet cible (une canette rouge).
3. **Orchestration Unifiée** : Connecter le sous-système de navigation et le sous-système de manipulation via des signaux de synchronisation ROS 2 ('/trailer/docking_status' et '/panda/task_completed'). **Validation**
Valider l'efficacité de l'agent RL en mesurant l'évolution de la courbe de récompense (passant de 0.8914.34) et assurer un

2 Architecture Système et Stacks Technologiques

L'architecture repose sur deux piliers technologiques complémentaires : l'écosystème ROS 2 pour la communication distribuée et le framework Robosuite/MuJoCo pour la simulation physique et l'apprentissage RL.

TABLE 1 – Synthèse des Stacks et Outils Logiciels Utilisés

Composant	Technologie / Bibliothèque	Rôle dans le projet
Système d'Exploitation	Linux Ubuntu 22.04 LTS	OS hôte pour la simulation et ROS 2
Middleware Robotique	ROS 2 Humble Hawksbill	Orchestration, TF2, Topics & Actions
Navigation Autonome	Nav2 (Planner, Controller, BT)	Planification de trajectoire du chariot
Localisation & TF	'robot_localization' (EKF)	Fusion de données IMU/GPS/Odométrie
Moteur de Simulation	MuJoCo / Robosuite 1.5.2	Physique de contact et dynamique du bras
Apprentissage RL	Stable-Baselines3	Implémentation des algorithmes PPO et SAC
Calcul Numérique	PyTorch (Version CPU)	Réseaux de neurones (Acteur-Critique)
Visualisation	RViz2 & GLFW Renderer	Visualisation 3D et suivi télémétrique

2.1 Chaîne de Transformations TF2

La synchronisation spatiale est garantie par la chaîne TF2 complète :

$$\text{world} \longrightarrow \text{map} \longrightarrow \text{odom} \longrightarrow \text{base_footprint} \longrightarrow \text{trailer_cart_drop_point}$$

Le repère racine du bras manipulateur ('world') est lié avec le repère de navigation Nav2 ('map') via un transformateur statique rigide à l'identité.

3 Pipeline d'Installation et Configuration du Workspace

Pour garantir l'isolation et la reproductibilité des environnements, un environnement virtuel Python dédié a été configuré dans ' /Downloads/robosuite-master/venv'.

3.1 Commandes d’Installation et de Dépendances

```
# 1. Activation du Venv
source /home/rabeb/Downloads/robosuite-master/venv/bin/activate

# 2. Installation de PyTorch CPU et Stable-Baselines3
pip install torch stable-baselines3 tensorboard \
  --index-url https://download.pytorch.org/whl/cpu \
  --extra-index-url https://pypi.org/simple

# 3. Configuration des Macros Robosuite
python /home/rabeb/Downloads/robosuite-master/robosuite/scripts/
  setup_macros.py

# 4. Compilation du workspace ROS 2
cd /home/rabeb/warehouse_fusion_ws
colcon build --packages-select warehouse_orchestrator
source install/setup.bash
```

Listing 1 – Installation des packages et du venv

4 Algorithmes et Stratégies d’Apprentissage (RL vs Déterministe)

Deux stratégies ont été développées et évaluées pour la tâche de *Pick & Place*.

4.1 1. Algorithme Proximal Policy Optimization (PPO)

L’agent RL apprend une politique stochastique $\pi_\theta(a|s)$ représentée par un réseau de neurones multilayer perceptron (MLP). L’objectif optimisé par PPO est donné par :

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

où $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ est le ratio de probabilité et $\hat{A}_t$ est l’avantage estimé par le réseau Critique.

4.1.1 Résultats Expérimentaux de l’Entraînement

Le modèle a été entraîné sur **200 000 timesteps** (durée : 32.66 minutes sur CPU).

TABLE 2 – Évolution des Métriques RL entre 50k et 200k pas d’apprentissage

Métrique	50 000 Pas	200 000 Pas	Interprétation
Récompense moyenne (<i>‘ep_reward_mean’</i>)	0.89	5.28	Hausse spectaculaire des performances
Récompense maximale observée	1.86	14.34	Saisie et levage réussis
Variance expliquée (<i>‘explained_variance’</i>)	-0.289	+0.711	Excellente prédiction de la fonction de récompense
Écart-type des actions (<i>‘std’</i>)	0.917	0.630	Mouvements ciblés et réduction de la variance
Fréquence d’entraînement (<i>‘fps’</i>)	104	102	Stabilité du moteur MuJoCo

4.2 2. Contrôleur Déterministe à Machine à Etats (IK)

En complément du modèle RL, un contrôleur cinématique déterministe à 8 états a été conçu pour garantir un taux de succès de 100% lors de la démonstration industrielle :

1. **MOVE_ABOVE** : Alignement XY au-dessus de la canette rouge ($z + 0.15\text{m}$).
2. **DESCEND** : Descente verticale à la hauteur de saisie ($z - 0.01\text{m}$).
3. **GRASP** : Maintien de la position et fermeture de la pince ($a_{gripper} = +1.0$).
4. **LIFT** : Soulevé vertical de sécurité ($z + 0.22\text{m}$).
5. **MOVE_TO_BIN** : Translation horizontale vers le bac du chariot amarré.
6. **LOWER** : Descente contrôlée dans le bac ($z_{bin} + 0.10\text{m}$).
7. **RELEASE** : Ouverture de la pince ($a_{gripper} = -1.0$) et libération.
8. **DONE** : Tâche achevée, envoi du signal `/panda/task_completed`.

5 Intégration Globale et Validation Expérimentale

L'intégration unifiée a été concrétisée par le script d'orchestration globale :

```
run_full_warehouse_simulation.py
```

5.1 Validation de la Mission Complète

Lors de l'exécution finale, la séquence d'événements s'est déroulée sans erreur :

- **PHASE 1 (Nav2)** : Le chariot autonome quitte le départ entrepôt (0.0, 0.0), franchit le corridor principal (-0.5, 1.2), puis atteint la pose d'amarrage (-0.24m, 2.80m).
- **PHASE 2 (Signalisation)** : Émission de la notification `/trailer/docking_status = True`. **PHASE 3 (Panda)** : Émission de la notification `LerobotPandaTectelacanetterougeauxcoordonnes[0.076, -0.098, 0.860], excutelasaisieettransbordel'objets[0.28, 0.80]`.
- **PHASE 4 (Succès)** : Émission du signal `/panda/task_completed = True` et affichage du message : **MISSION ACCOMPLIE**.

6 Conclusion et Perspectives

Le projet **Warehouse Fusion** a démontré avec succès l'intégration synergique de la navigation autonome sous ROS 2 et de la manipulation robotique avancée par Apprentissage par Renforcement et logique déterministe sous Robosuite.

Les résultats obtenus (récompense RL grimant à 14.34, `explained_variance` de 0.711, `et simulation unifiée` 100%).

Sim-to-Real Transfer : Déploiement des politiques RL entraînées sur un véritable bras Franka Emika Panda physique.

Vision Basée sur la Caméra (RGB-D) : Entraînement de l'agent directement sur des observations visuelles (`use_camera_obs = True`). **Algorithme SAC (Soft Actor-Critic)** : Exploration de l'efficacité spectrale de SAC sur des horizons d'entraînement plus longs (500 000+ pas).